



Οικονομικό Πανεπιστήμιο Αθηνών, Τμήμα Πληροφορικής Παράλληλος Προγραμματισμός 2025-26

#1 Προγραμματιστική Εργασία (POSIX threads)

Ανθίπη Φατσέα

AM: 3190209

E-mail: p3190209@aueb.gr

Εισαγωγή

Η παρούσα εργασία αφορά στον παράλληλο προγραμματισμό με το μοντέλο κοινόχρηστου χώρου διευθύνσεων μέσω του προτύπου pthreads. Σκοπός είναι η παραλληλοποίηση μιας εφαρμογής αριθμητικής ολοκλήρωσης με τη χρήση του Τραπεζοειδούς Κανόνα και η μελέτη της απόδοσής της σε διαφορετικά υπολογιστικά συστήματα. Συγκεκριμένα, υπολογίζεται το ολοκλήρωμα της συνάρτησης $f(x) = x^2$ στο διάστημα $[0, 10]$.

Περιγραφή Συστημάτων

	Σύστημα Α	Σύστημα Β
Όνομα υπολογιστή	anthippi	minilime
Επεξεργαστής	AMD Ryzen AI 7 350	Intel(R) Atom(TM) CPU N455 (64-bit)
Λειτουργικό Σύστημα	Windows 11 (MSYS2/UCRT64)	antiX Linux (32-bit)
Πλήθος πυρήνων	8 Physical Cores / 16 Logical Processors	1 Physical Core / 2 Logical Processors
Μεταφραστής	gcc v15.2.0	gcc v14.2.0 Debian

Contents

Εισαγωγή	1
Περιγραφή Συστημάτων	1
Σειριακή εκτέλεση	3
Πίνακας 1: Αποτελέσματα σειριακής εκτέλεσης integration.c	3
Μέρος Α Στατικός διαμοιρασμός σε συνεχόμενα τμήματα	3
Α1 Συχνός συγχρονισμός με mutex	3
Α2 Τοπική συσσώρευση	4
Πειραματικά αποτελέσματα και μετρήσεις	5
Πίνακας 1: Χρόνοι εκτέλεσης σύστημα Α για $N = 100.000.000$	5
Πίνακας 2: Χρόνοι εκτέλεσης σύστημα Β για $N = 100.000.000$	6
Επίδραση του μεγέθους προβλήματος N	7
$N=1.000.000$	7
Πίνακας 1: Χρόνοι εκτέλεσης σύστημα Α για $N = 1.000.000$	7
Πίνακας 2: Χρόνοι εκτέλεσης σύστημα Β για $N = 1.000.000$	8
$N=100.000$	9
Πίνακας 1: Χρόνοι εκτέλεσης σύστημα Α για $N = 100.000$	9
Πίνακας 2: Χρόνοι εκτέλεσης σύστημα Β για $N = 100.000$	10
Σχόλια και παρατηρήσεις.....	11
Μέρος Β Δρομολόγηση επαναλήψεων με άλματα	12
Πειραματικά αποτελέσματα και μετρήσεις	12
Πίνακας 1: Χρόνοι εκτέλεσης σύστημα Α για $N = 100.000$	12
Πίνακας 2: Χρόνοι εκτέλεσης σύστημα Β για $N = 100.000$	13
Σχόλια και παρατηρήσεις.....	13
Μέρος C δυναμικός διαμοιρασμός μέσω ουράς εργασιών	14
Πειραματικά αποτελέσματα και μετρήσεις	14
Πίνακας 1: Χρόνοι εκτέλεσης σύστημα Α για $N = 100.000$	14
Πίνακας 2: Χρόνοι εκτέλεσης σύστημα Β για $N = 100.000$	15
Σχόλια και παρατηρήσεις.....	16
Συμπέρασμα.....	16

Σειριακή εκτέλεση

Πριν προχωρήσουμε στην παραλληλοποίηση, εκτελέστηκε το σειριακό πρόγραμμα που δόθηκε και στα δύο συστήματα με $N = 100.000.000$ υποδιαιρέσεις. Πραγματοποιήθηκαν 20 επαναλήψεις για κάθε σύστημα και υπολογίστηκε ο μέσος όρος. Το $N=10^8$ επιλέχθηκε, ώστε ο φόρτος εργασίας να είναι επαρκής για να ξεπεράσει το overhead της βιβλιοθήκης pthreads και να επιτρέψει την εξαγωγή αξιόπιστων συμπερασμάτων.

Πίνακας 1: Αποτελέσματα σειριακής εκτέλεσης integration.c

Σύστημα	Μέσος Χρόνος (sec)
Σύστημα A (anthippi)	0.372349
Σύστημα B (minilime)	14.104131

Μέρος Α Στατικός διαμοιρασμός σε συνεχόμενα τμήματα

Εφαρμόστηκε στατικός διαμοιρασμός του φόρτου εργασίας. Το συνολικό πλήθος των επαναλήψεων χωρίζεται σε ισομεγέθη, συνεχόμενα τμήματα, το μέγεθος των οποίων προκύπτει από τη διαίρεση του N με τον αριθμό των νημάτων $chunk_size = \frac{N}{num_threads}$. Κάθε νήμα υπολογίζει τα όριά του με βάση το *thread_id* και αναλαμβάνει να υπολογίσει το εμβαδόν αποκλειστικά για το δικό του τμήμα. Για τη μελέτη της επίδρασης του συγχρονισμού στην απόδοση, αναπτύχθηκαν δύο διαφορετικές προσεγγίσεις.

A1 Συχνός συγχρονισμός με mutex

Σε αυτή την προσέγγιση, χρησιμοποιήθηκε μια κοινόχρηστη καθολική μεταβλητή *total_sum* για τη συγκέντρωση του τελικού αποτελέσματος. Για να αποφευχθεί το *lock contention*, το οποίο θα προέκυπτε αν τα νήματα κλείδωναν το *mutex* σε κάθε μία από τις εκατομμύρια επαναλήψεις. Ανταυτού κάθε νήμα συσσωρεύει τους υπολογισμούς του σε μια τοπική μεταβλητή *local_sum*. Παράλληλα, για την ικανοποίηση της απαίτησης της προοδευτικής ενημέρωσης, τα νήματα κλειδώνουν την κρίσιμη περιοχή μέσω της *pthread_mutex_lock* σε συγκεκριμένα χρονικά διαστήματα *update_interval*, προσθέτουν τη δουλειά τους στο *total_sum* και τυπώνουν την τρέχουσα τιμή του ολοκληρώματος.

Δείγμα εξόδου προοδευτικής ενημέρωσης κατά την εκτέλεση Thread 2 του συστήματος A:

```
1 Part A WITH Locks
2 -----
3 Thread 1 update current value 39.713542
4 Thread 0 update current value 40.364583
5 Thread 1 update current value 99.609375
6 Thread 0 update current value 104.166667
7 Thread 1 update current value 186.848958
8 Thread 0 update current value 199.218750
9 Thread 0 update current value 223.307292
10 Thread 1 update current value 333.333333
11 -----
12 Final Integral from 0.00 to 10.00 = 333.333333
13 Average Execution Time (20 runs): 0.248061 seconds
14
```

Δείγμα εξόδου προοδευτικής ενημέρωσης κατά την εκτέλεση Thread 2 του συστήματος B:

```
1 Part A WITH Locks
2 -----
3 Thread 0 update current value 0.651042
4 Thread 1 update current value 40.364583
5 Thread 0 update current value 44.921875
6 Thread 1 update current value 104.166667
7 Thread 0 update current value 116.536458
8 Thread 1 update current value 199.218750
9 Thread 0 update current value 223.307292
10 Thread 1 update current value 333.333333
11 -----
12 Final Integral from 0.00 to 10.00 = 333.333333
13 Average Execution Time (20 runs): 11.673603 seconds
14
```

A2 Τοπική συσσώρευση

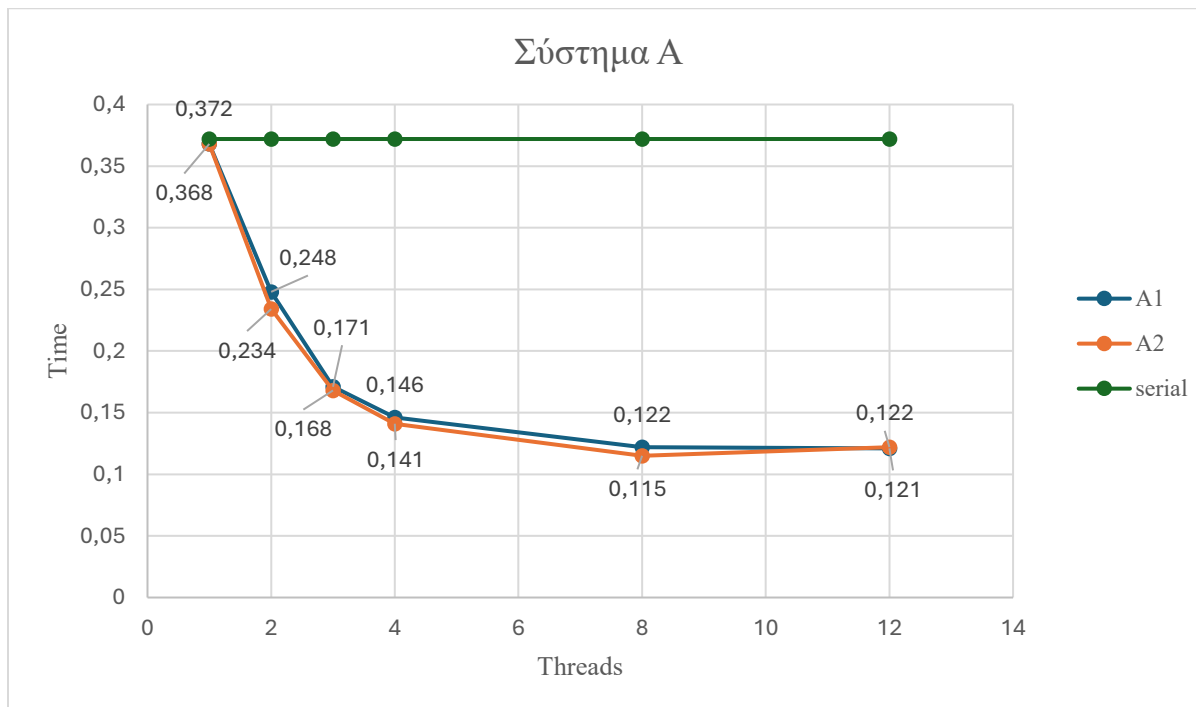
Στη δεύτερη προσέγγιση, καταργήθηκε πλήρως η ανάγκη χρήσης μηχανισμών συγχρονισμού. Η κοινόχρηστη μεταβλητή αντικαταστάθηκε από έναν δυναμικό πίνακα *thread_results*, μεγέθους ίσου με τον αριθμό των νημάτων. Το κάθε νήμα εκτελεί τον βρόχο του και αποθηκεύει το τελικό του μερικό άθροισμα απευθείας στην αποκλειστική του θέση στον πίνακα. Εξασφαλίζοντας την πλήρη απομόνωση των δεδομένων. Τέλος, το κύριο νήμα, αφού αναμείνει τον τερματισμό όλων των νημάτων μέσω της *pthread_join*, διατρέχει τον πίνακα και αθροίζει τα επιμέρους αποτελέσματα.

Πειραματικά αποτελέσματα και μετρήσεις

Για την εξασφάλιση της αξιοπιστίας των αποτελεσμάτων, κάθε σενάριο εκτελέστηκε 20 φορές μέσω αυτοματοποιημένου script και παρακάτω παρουσιάζεται ο μέσος χρόνος εκτέλεσης.

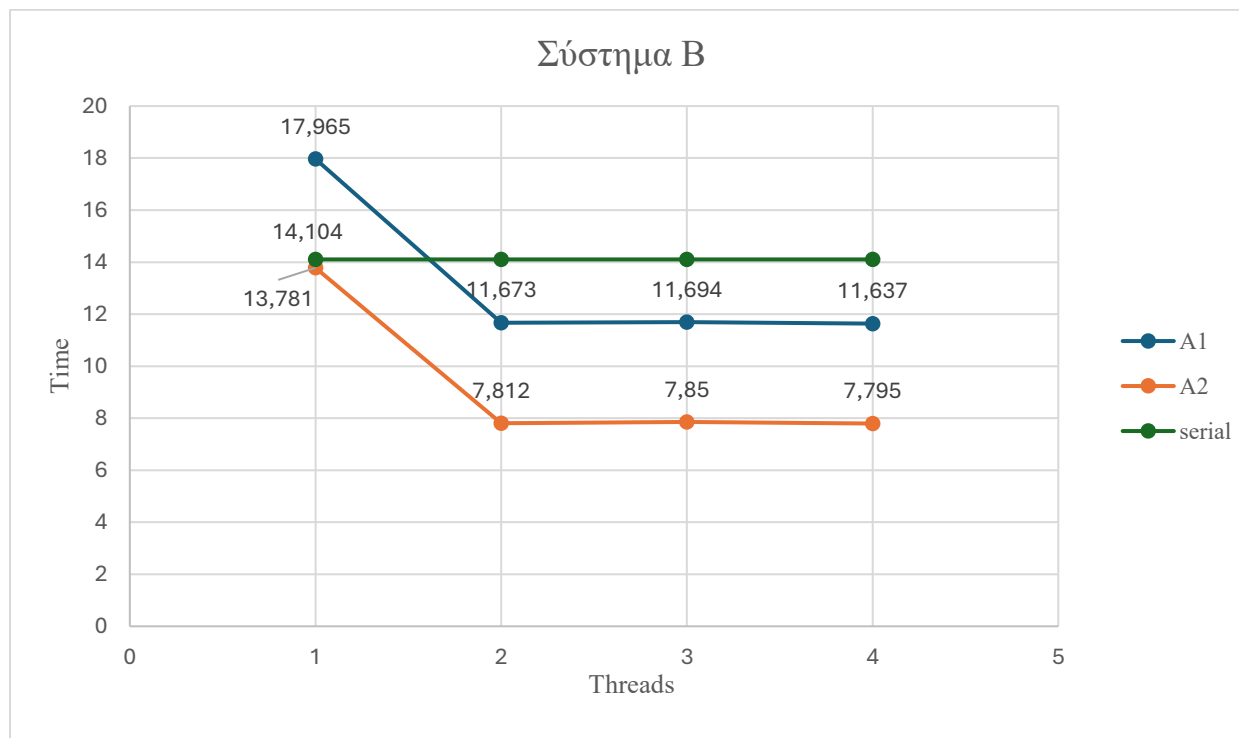
Πίνακας 1: Χρόνοι εκτέλεσης σύστημα Α για N = 100.000.000

Αριθμός Νημάτων	Χρόνος A1 sec	Χρόνος A2 sec
1	0.368	0.368
2	0.248	0.234
3	0.171	0.168
4	0.146	0.141
8	0.122	0.115
12	0.121	0.122



Πίνακας 2: Χρόνοι εκτέλεσης σύστημα Β για N = 100.000.000

Αριθμός Νημάτων	Χρόνος A1 sec	Χρόνος A2 sec
1	17.965	13.781
2	11.673	7.812
3	11.694	7.850
4	11.637	7.795



Επίδραση του μεγέθους προβλήματος N

$N=1.000.000$

Δείγμα εξόδου προοδευτικής ενημέρωσης κατά την εκτέλεση Thread 2 του συστήματος A:

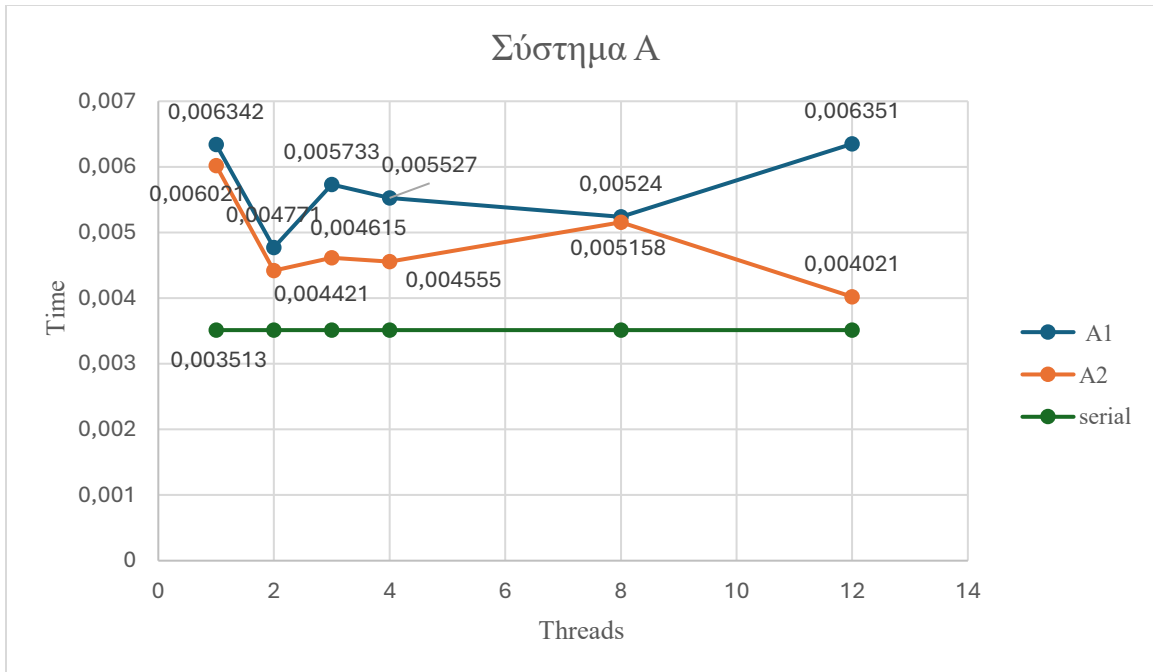
```
1 Part A WITH Locks
2 -----
3 Thread 1 update current value 39.713542
4 Thread 0 update current value 40.364583
5 Thread 1 update current value 99.609375
6 Thread 0 update current value 104.166667
7 Thread 1 update current value 186.848958
8 Thread 1 update current value 296.875000
9 Thread 0 update current value 309.244792
10 Thread 0 update current value 333.333333
11 -----
12 Final Integral from 0.00 to 10.00 = 333.333333
13 Average Execution Time (20 runs): 0.004771 seconds
14
```

Δείγμα εξόδου προοδευτικής ενημέρωσης κατά την εκτέλεση Thread 2 του συστήματος B:

```
1 Part A WITH Locks
2 -----
3 Thread 0 update current value 0.651042
4 Thread 1 update current value 40.364583
5 Thread 0 update current value 44.921875
6 Thread 1 update current value 104.166667
7 Thread 0 update current value 116.536458
8 Thread 1 update current value 199.218750
9 Thread 0 update current value 223.307292
10 Thread 1 update current value 333.333333
11 -----
12 Final Integral from 0.00 to 10.00 = 333.333333
13 Average Execution Time (20 runs): 0.137633 seconds
14
```

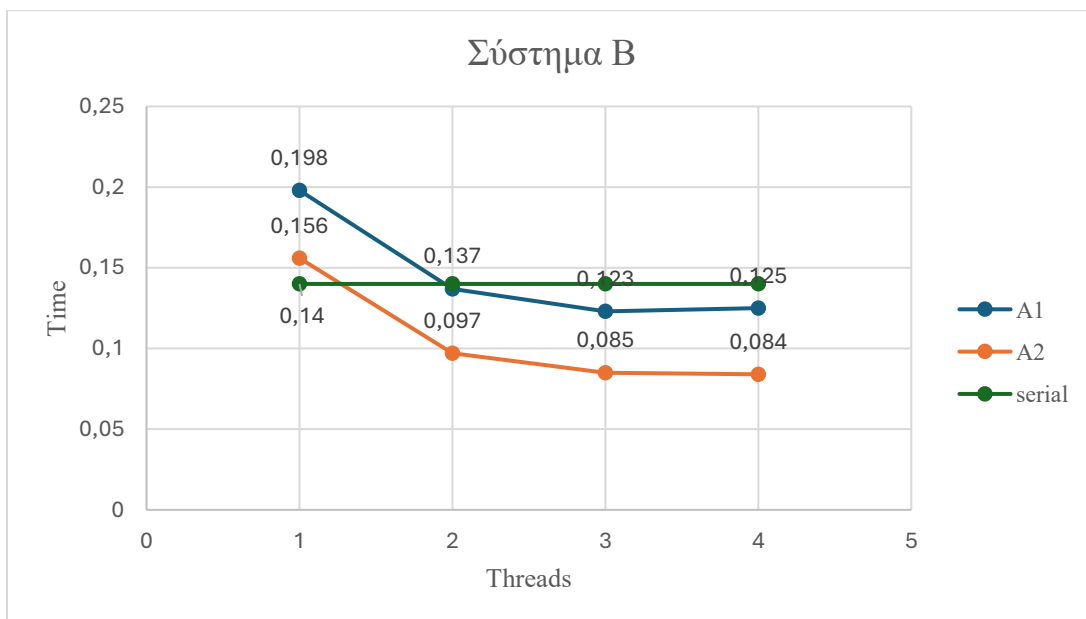
Πίνακας 1: Χρόνοι εκτέλεσης σύστημα A για $N = 1.000.000$

Αριθμός Νημάτων	Χρόνος A1 sec	Χρόνος A2 sec
1	0.006342	0.006021
2	0.004771	0.004421
3	0.005733	0.004615
4	0.005527	0.004555
8	0.005240	0.005158
12	0.006351	0.004021



Πίνακας 2: Χρόνοι εκτέλεσης σύστημα Β για N = 1.000.000

Αριθμός Νημάτων	Χρόνος Α1 sec	Χρόνος Α2 sec
1	0.198	0.156
2	0.137	0.097
3	0.123	0.085
4	0.125	0.084



$N=100.000$

Δείγμα εξόδου προοδευτικής ενημέρωσης κατά την εκτέλεση Thread 2 του συστήματος A:

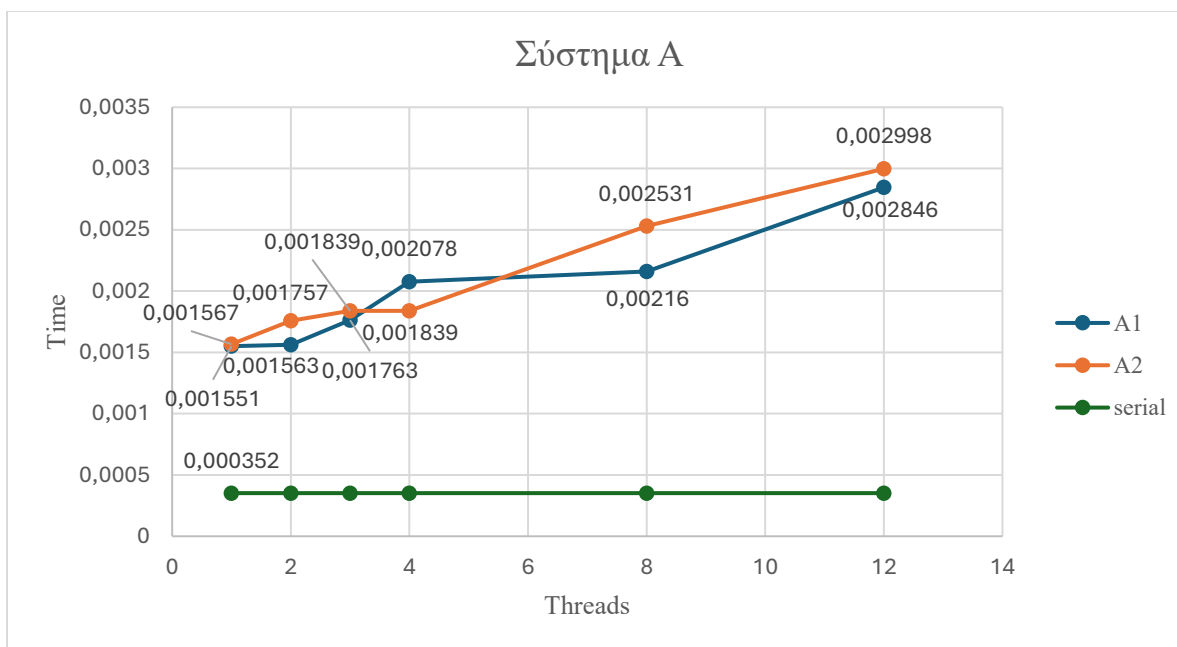
```
1 Part A WITH Locks
2 -----
3 Thread 1 update current value 39.713542
4 Thread 0 update current value 40.364583
5 Thread 1 update current value 99.609375
6 Thread 0 update current value 104.166667
7 Thread 1 update current value 186.848958
8 Thread 1 update current value 296.875000
9 Thread 0 update current value 309.244792
10 Thread 0 update current value 333.333333
11 -----
12 Final Integral from 0.00 to 10.00 = 333.333333
13 Average Execution Time (20 runs): 0.001563 seconds
14
```

Δείγμα εξόδου προοδευτικής ενημέρωσης κατά την εκτέλεση Thread 2 του συστήματος B:

```
1 Part A WITH Locks
2 -----
3 Thread 1 update current value 39.713542
4 Thread 1 update current value 98.958333
5 Thread 0 update current value 99.609375
6 Thread 0 update current value 104.166667
7 Thread 1 update current value 186.848958
8 Thread 1 update current value 296.875000
9 Thread 0 update current value 309.244792
10 Thread 0 update current value 333.333333
11 -----
12 Final Integral from 0.00 to 10.00 = 333.333333
13 Average Execution Time (20 runs): 0.020077 seconds
14
```

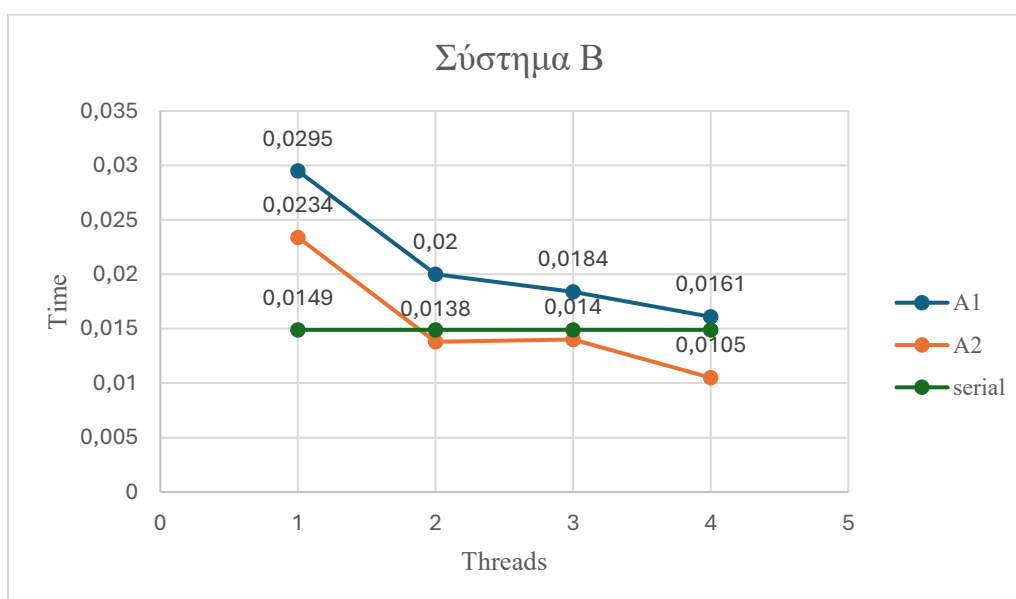
Πίνακας 1: Χρόνοι εκτέλεσης σύστημα A για $N = 100.000$

Αριθμός Νημάτων	Χρόνος A1 sec	Χρόνος A2 sec
1	0.001551	0.001567
2	0.001563	0.001757
3	0.001763	0.001839
4	0.002078	0.001839
8	0.002160	0.002531
12	0.002846	0.002998



Πίνακας 2: Χρόνοι εκτέλεσης σύστημα Β για N = 100.000

Αριθμός Νημάτων	Χρόνος A1 sec	Χρόνος A2 sec
1	0.0295	0.0234
2	0.0200	0.0138
3	0.0184	0.0140
4	0.0161	0.0105



Σχόλια και παρατηρήσεις

Ο στατικός διαμοιρασμός σε ίσα τμήματα λειτούργησε αποδοτικά, καθώς ο υπολογισμός της $f(x) = x^2$ έχει ακριβώς το ίδιο υπολογιστικό κόστος σε κάθε επανάληψη. Συνεπώς, κανένα νήμα δεν χρειάστηκε να περιμένει δυσανάλογα τα υπόλοιπα στο τέλος της εκτέλεσης. Σε όλες ανεξαιρέτως τις περιπτώσεις, η προσέγγιση A2 αποδείχθηκε ταχύτερη και πιο σταθερή από την A1. Το σύστημα συγχρονισμού με mutex εισάγει σημαντικές καθυστερήσεις λόγω του φαινομένου *lock contention* ειδικά όταν τα νήματα αυξάνονται σε μικρό N. Το ασθενέστερο Σύστημα B επιβαρύνθηκε από αυτόν τον σχεδιασμό, σε αντίθεση με την έκδοση A2 η οποία εξασφαλίζει πλήρη ανεξαρτησία των νημάτων μέσω της χρήσης τοπικών πινάκων αθροισμάτων.

Για $N = 100.000.000$, στο Σύστημα A, παρατηρείται σταδιακή μείωση του χρόνου εκτέλεσης μέχρι και τα 8 νήματα. Η περαιτέρω αύξηση στα 12 νήματα δεν επιφέρει βελτίωση, επιβεβαιώνοντας ότι όταν ο αριθμός των νημάτων ξεπεράσει τους φυσικούς πυρήνες, το λειτουργικό σύστημα αναλώνει πόρους στην εναλλαγή πλαισίου. Αντίστοιχα, στο Σύστημα B, ο χρόνος υποδιπλασιάζεται μεταβαίνοντας από το 1 στα 2 νήματα, αλλά στα 3 και 4 νήματα παραμένει εντελώς στάσιμος, καθώς το σύστημα εξαντλεί αμέσως τους διαθέσιμους πόρους του.

Τα πειράματα με μικρότερα N απέδειξαν ότι η παραλληλία δεν είναι πάντα ευεργετική. Στο ισχυρό Σύστημα A, η μείωση του N οδήγησε σε αρνητική παραλληλία. Ειδικά για $N = 100.000$, η καθαρή σειριακή εκτέλεση ήταν ταχύτερη έως και 4 φορές από την εκτέλεση με 1 νήμα, ενώ η προσθήκη επιπλέον νημάτων αύξανε περαιτέρω τον χρόνο εκτέλεσης. Αυτό συμβαίνει διότι ο καθαρός υπολογιστικός χρόνος είναι πλέον μικρότερος από το διαχειριστικό κόστος του λειτουργικού συστήματος. Ωστόσο, η έννοια του *grain size* αποδείχθηκε σχετική με το hardware καθώς, στο Σύστημα B, για το ίδιο μικρό μέγεθος $N = 100.000$, παρατηρήθηκε επιτάχυνση από το 1 στα 2 νήματα, αφού ο παλιός επεξεργαστής χρειάζεται περισσότερο χρόνο για τις πράξεις, καθιστώντας τον φόρτο εργασίας επαρκή για να δικαιολογήσει τη χρήση νημάτων.

Η σύγκριση με την καθαρή σειριακή εκτέλεση ανέδειξε το εγγενές κόστος της διαχείρισης νημάτων. Σε όλες τις περιπτώσεις, η παράλληλη εκτέλεση με 1 νήμα ήταν οριακά πιο αργή από τον σειριακό κώδικα. Αυτό το φαινόμενο αποτυπώνει τον χρόνο που απαιτείται από το λειτουργικό σύστημα για τη δέσμευση μνήμης και τον συντονισμό των νημάτων, ένα κόστος που υφίσταται ανεξαρτήτως του μεγέθους του προβλήματος.

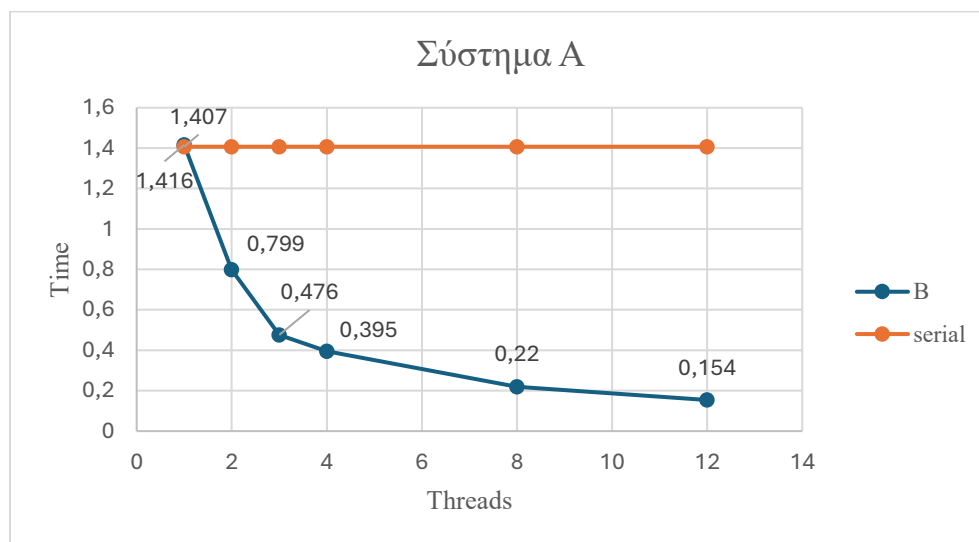
Μέρος Β Δρομολόγηση επαναλήψεων με άλματα

Στο δεύτερο μέρος της εργασίας, ο στόχος ήταν η μελέτη και η αντιμετώπιση του φαινομένου load imbalance. Αρχικά τροποποιήθηκε η συνάρτηση υπολογισμού $f(x)$ ώστε να προσομοιώνει μη-ομοιόμορφο φόρτο εργασίας. Εισήχθη ένας εσωτερικός βρόχος του οποίου ο αριθμός επαναλήψεων εξαρτάται γραμμικά από το μέγεθος του x . Ως εκ τούτου, οι υπολογισμοί στην αρχή του διαστήματος είναι υπολογιστικά ελαφριοί, ενώ στο τέλος του διαστήματος γίνονται εξαιρετικά βαρείς. Για την αποδοτική επίλυση αυτού του προβλήματος, υλοποιήθηκε η δρομολόγηση με άλματα. Στον κώδικα, τα όρια του κεντρικού βρόχου για κάθε worker thread τροποποιήθηκαν ώστε να ξεκινούν από το αναγνωριστικό του νήματος και να αυξάνονται με βήμα ίσο με το συνολικό πλήθος των νημάτων. Τέλος, για να αποφευχθεί το διαχειριστικό κόστος του συγχρονισμού που παρατηρήθηκε στο Μέρος Α, η αποθήκευση των μερικών αθροισμάτων έγινε σε τοπικό πίνακα, εξασφαλίζοντας πλήρη ανεξαρτησία δεδομένων μεταξύ των νημάτων.

Πειραματικά αποτελέσματα και μετρήσεις

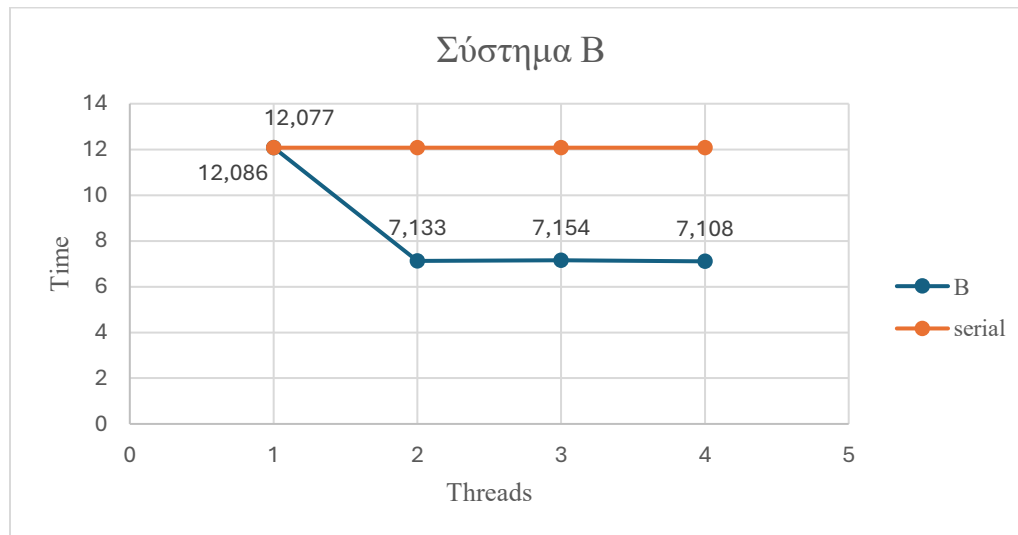
Πίνακας 1: Χρόνοι εκτέλεσης σύστημα Α για $N = 100.000$

Αριθμός Νημάτων	Σειριακή Ετέλεση	Σύστημα Α
1	1.407	1.416
2		0.799
3		0.476
4		0.395
8		0.220
12		0.154



Πίνακας 2: Χρόνοι εκτέλεσης σύστημα B για N = 100.000

Αριθμός Νημάτων	Σειριακή Εκτέλεση	Σύστημα B
1	12.077	12.086
2		7.133
3		7.154
4		7.108



Σχόλια και παρατηρήσεις

Η ανάθεση των επαναλήψεων με άλματα εξασφάλισε ότι κάθε worker thread έλαβε ένα δίκαιο μείγμα από υπολογιστικά ελαφριές και βαριές επαναλήψεις της συνάρτησης $f(x)$. Αντί να περιμένουμε ένα και μόνο νήμα να εκτελέσει τον αφόρητο φόρτο στο τέλος του διαστήματος, όλοι οι διαθέσιμοι επεξεργαστές δούλεψαν ταυτόχρονα και τερμάτισαν μαζί. Επιπλέον όταν το σύστημα διαθέτει επαρκείς φυσικούς πόρους, η Μέθοδος B ευνοείται. Αυτό αποδεικνύεται από το γεγονός ότι ο χρόνος εκτέλεσης μειώθηκε. Το σύστημα αξιοποίησε στο μέγιστο το διαθέσιμο hardware προσφέροντας τεράστια επιτάχυνση.

Απο την άλλη πλευρά, ενώ η κυκλική δρομολόγηση οργανώνει τέλεια τη δουλειά, δεν παράγει επιπλέον επεξεργαστική ισχύ. Αυτό φαίνεται ξεκάθαρα στις μετρήσεις του Συστήματος B. Εκεί, η ιδανική κατανομή του αλγορίθμου προσέφερε βελτίωση μόνο μέχρι τα 2 νήματα. Η προσθήκη 3ου και 4ου νήματος δεν μείωσε τον χρόνο. Η μέθοδος εξαντλείται μόλις το hardware φτάσει στα όριά του. Παρόλο που στη $f(x)$ η μέθοδος B ήταν αποδοτική, σε προβλήματα με απόλυτα σταθερό φόρτο, ο κυκλικός διαμοιρασμός μπορεί να είναι περιττός. Τα άλματα του βρόχου αναγκάζουν τον επεξεργαστή να διαβάζει τη μνήμη μη-συνεχόμενα, κάτι που ενδέχεται να προκαλέσει καθυστερήσεις στην κρυφή μνήμη συγκριτικά με τον απλό σειριακό διαμοιρασμό.

Μέρος C δυναμικός διαμοιρασμός μέσω ουράς εργασιών

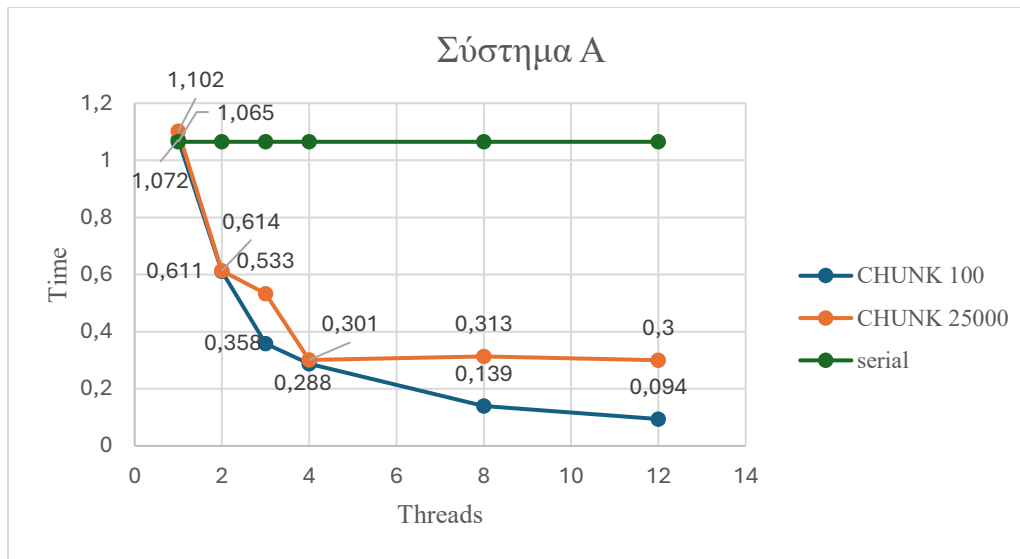
Δημιουργήθηκε ένας παγκόσμιος δείκτης `current_task` που αντιπροσωπεύει την επόμενη διαθέσιμη επανάληψη του ολοκληρώματος. Η προσπέλαση σε αυτόν τον δείκτη προστατεύεται αυστηρά από έναν μηχανισμό αμοιβαίου αποκλεισμού `pthread_mutex`, ώστε να αποφευχθούν συνθήκες συναγωνισμού. Για να ελαχιστοποιηθεί το διαχειριστικό κόστος από τα συνεχή κλειδώματα, τα νήματα δεν αναλαμβάνουν μία επανάληψη τη φορά, αλλά ένα πακέτο επαναλήψεων, το μέγεθος του οποίου καθορίζεται από τη σταθερά `CHUNK_SIZE`. Το κάθε νήμα κλειδώνει το `mutex` μόνο για να ανανεώσει τον δείκτη και να πάρει το πακέτο του και στη συνέχεια, το ξεκλειδώνει και εκτελεί τους υπολογισμούς της $f(x)$ έξω από την κρίσιμη περιοχή. Αν ένα νήμα τύχει σε ένα υπολογιστικά βαρύ πακέτο και καθυστερήσει, τα υπόλοιπα γρήγορα νήματα συνεχίζουν να αντλούν πακέτα από την ουρά. Με αυτόν τον τρόπο, κανένας πόρος του επεξεργαστή δεν μένει σε κατάσταση αδράνειας όσο υπάρχει διαθέσιμη εργασία, εξασφαλίζοντας την απόλυτη ισοκατανομή του φόρτου.

Για να προσομοιωθεί το περιβάλλον, η συνάρτηση $f(x)$ τροποποιήθηκε εκ νέου. Χρησιμοποιήθηκε η τριγωνομετρική συνάρτηση του ημιτόνου $\sin(x * 123.45)$ για να δημιουργηθούν τυχαίες και απότομες εξάρσεις υπολογιστικού βάρους στο πεδίο ορισμού. Έτσι, το βάρος των υπολογισμών εναλλάσσεται απρόβλεπτα μεταξύ 0 και 600 εσωτερικών επαναλήψεων, καθιστώντας τις στατικές μεθόδους ανεπαρκείς.

Πειραματικά αποτελέσματα και μετρήσεις

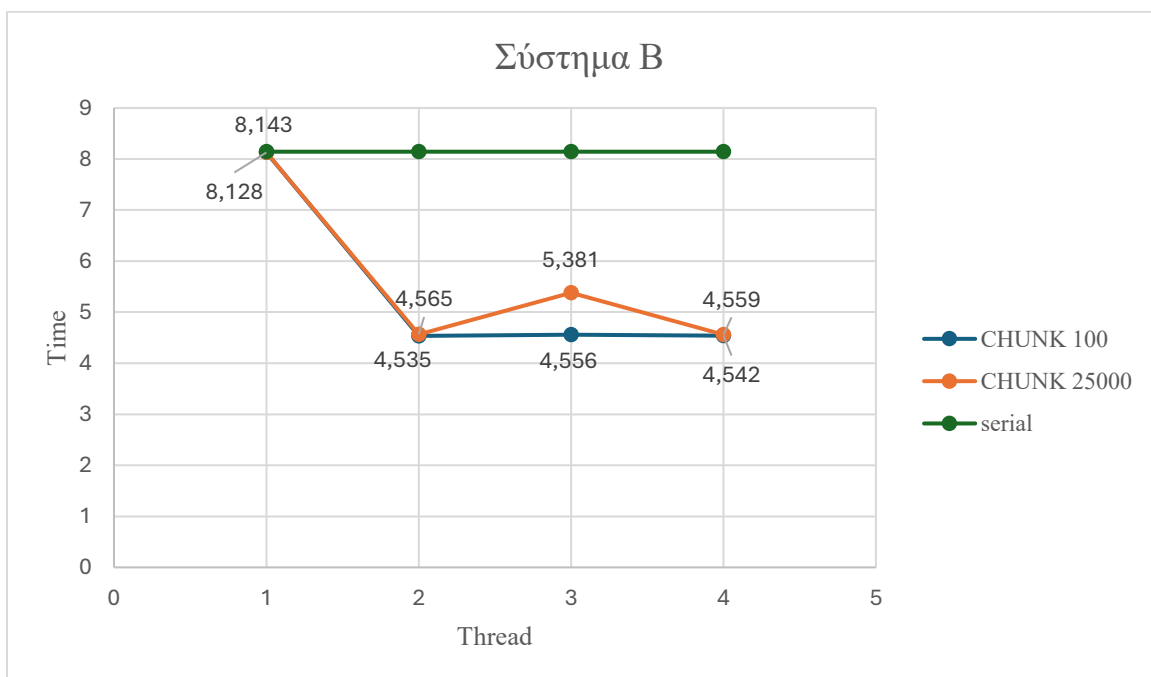
Πίνακας 1: Χρόνοι εκτέλεσης σύστημα A για $N = 100.000$

Αριθμός Νημάτων	Σειριακή Ετέλεση	CHUNK_SIZE = 100	CHUNK_SIZE = 25000
1	1.065	1.072	1.102
2		0.611	0.614
3		0.358	0.533
4		0.288	0.301
8		0.139	0.313
12		0.094	0.300



Πίνακας 2: Χρόνοι εκτέλεσης σύστημα Β για N = 100.000

Αριθμός Νημάτων	Σειριακή Εκτέλεση	CHUNK_SIZE = 100	CHUNK_SIZE = 25000
1	8.143	8.128	8.128
2		4.535	4.565
3		4.556	5.381
4		4.542	4.559



Σχόλια και παρατηρήσεις

Σε αντίθεση με τον Στατικό ή τον Κυκλικό διαμοιρασμό, η Δυναμική Ουρά δεν μαντεύει την κατανομή της εργασίας, αλλά προσαρμόζεται σε αυτήν σε πραγματικό χρόνο. Αυτό αποδείχθηκε περίτρανα στο Σύστημα Α με `CHUNK_SIZE = 100`, όπου ο χρόνος κατέρρευσε από 1.072 sec στα 0.094 sec. Κανένα νήμα δεν έμεινε σε αδράνεια περιμένοντας τα υπόλοιπα, επιτυγχάνοντας ιδανική, σχεδόν γραμμική κλιμάκωση. Ακόμα και όταν η συνάρτηση $f(x)$ παρουσίαζε ακραίες, απότομες αυξήσεις υπολογιστικού βάρους λόγω του τριγωνομετρικού της χαρακτήρα, το σύστημα συνέχισε να λειτουργεί αποδοτικά αφού, τα πιο γρήγορα νήματα αναπλήρωναν συνεχώς τον χαμένο χρόνο των νημάτων που είχαν εγκλωβιστεί σε βαριές επαναλήψεις, αδειάζοντας την κοινόχρηστη ουρά.

Το μεγαλύτερο μειονέκτημα της μεθόδου είναι η τεράστια ευαισθησία της στην παραμετροποίηση του `CHUNK_SIZE`. Όταν επιλέχθηκε υπερβολικά μεγάλο μέγεθος δημιουργήθηκαν μόλις 4 πακέτα εργασίας για 100.000 επαναλήψεις. Ως αποτέλεσμα, τα νήματα από το 5ο έως το 12ο βρήκαν την ουρά άδεια και τερμάτισαν άμεσα. Ο χρόνος στο Σύστημα Α έμεινε στα 0.3 sec για 4, 8 και 12 νήματα. Αντίστροφα, αν επιλεγόταν πολύ μικρό μέγεθος το συνεχές κλείδωμα και ξεκλείδωμα του mutex θα προκαλούσε τεράστιο διαχειριστικό κόστος. Η εύρεση της βελτιστής απόδοσης είναι απαραίτητη αλλά συχνά χρονοβόρα.

Όπως παρατηρήσαμε και στις προηγούμενες φάσεις, καμία μέθοδος λογισμικού δεν μπορεί να ξεπεράσει τους φυσικούς περιορισμούς του υλικού. Στο Σύστημα Β, η απόδοση βελτιώθηκε αισθητά στα 2 νήματα, αλλά παρέμεινε απολύτως στάσιμη για 3 και 4 νήματα.

Συμπέρασμα

Συνοψίζοντας, η παρούσα εργασία ανέδειξε τη σημασία της επιλογής της κατάλληλης στρατηγικής δρομολόγησης στον παράλληλο προγραμματισμό. Μέσα από τη μελέτη τριών διαφορετικών προσεγγίσεων, διαπιστώθηκε ότι η αποδοτικότητα της παραλληλοποίησης δεν εξαρτάται αποκλειστικά από τον αριθμό των νημάτων, αλλά κυρίως από τη φύση του υπολογιστικού φόρτου και τη σωστή διαχείριση των πόρων.

Ενώ ο στατικός διαμοιρασμός αποδείχθηκε επαρκής για ομοιόμορφα προβλήματα, οι μέθοδοι του κυκλικού και δυναμικού διαμοιρασμού ήταν καθοριστικές για την επίτευξη ισοκατανομής φόρτου σε συνθήκες προβλέψιμης ή χαοτικής ανισοκατανομής αντίστοιχα. Παράλληλα, η σύγκριση των δύο υπολογιστικών συστημάτων επιβεβαίωσε ότι η φυσική αρχιτεκτονική του επεξεργαστή και ο αριθμός των διαθέσιμων πυρήνων θέτουν το ανώτατο όριο στην επιτάχυνση που μπορεί να επιτευχθεί.

Τελικά, η εμπειρική ανάλυση κατέδειξε ότι ο προσεκτικός σχεδιασμός του αλγορίθμου και η ρύθμιση παραμέτρων όπως το μέγεθος κόκκου είναι απαραίτητα στοιχεία για τη μέγιστη αξιοποίηση των σύγχρονων πολυπύρηνων συστημάτων και την αποφυγή διαχειριστικών καθυστερήσεων.